

Testing Guide - Phase 5A SaaS Installation Flow

Executive Summary

This guide provides a comprehensive, step-by-step testing plan for the **Qryx SaaS Installation Flow** implemented in Phase 5A. It covers all critical paths from initial installation to successful OAuth call-back.

Flow Overview:

Shopify Install → Shop Session → Login/Signup → Plan Selection → Stripe Checkout → OAuth → Dashboard

1. Pre-Test Checklist

1.1 Environment Setup

- [] **Local Dev Server Running:** `cd /home/ubuntu/jnx-os/nextjs_space && yarn dev`
- [] **Environment Variables Set:** Check `/home/ubuntu/jnx-os/nextjs_space/.env`
- [] **Database Connected:** Verify Supabase connection
- [] **Stripe Test Mode:** Start with test keys for initial testing
- [] **Clerk Dashboard:** Have access to verify user creation
- [] **Stripe Dashboard:** Have access to monitor checkouts

1.2 Required Credentials

Test Shopify Store:

- Store: `shopbotv3.myshopify.com`
- Admin URL: `https://admin.shopify.com/store/shopbotv3`

Test Clerk User:

- Email: `test@jnxlabs.ai`
- Password: `TestUser123!`

Test Stripe Card:

- Card Number: `4242 4242 4242 4242`
- Expiry: Any future date (e.g., 12/26)
- CVC: Any 3 digits (e.g., 123)
- ZIP: Any 5 digits (e.g., 12345)

1.3 Tools Needed

- **Browser:** Chrome/Firefox with DevTools open
- **Terminal:** For monitoring logs (`yarn dev`)
- **Database Client:** Supabase Dashboard or SQL editor
- **Stripe CLI** (optional): For local webhook testing

2. Test Scenario 1: New User - Complete Flow

Goal: Verify entire flow from Shopify install to dashboard access for a brand new user.

Step 1: Initiate Installation

Action:

1. Open browser
2. Navigate to: `https://www.jnqlabs.ai/api/qryx/install?shop=shopbotv3.myshopify.com`

Expected Result:

- ☒ Redirects to `/login` page
- ☒ URL contains no error messages
- ☒ Shop session created (check DevTools → Application → Cookies)

Verify Shop Session:

- Cookie Name: `shop_session`
- Cookie Value: JWT token (long string)
- Expiry: 30 minutes from creation

DevTools Console:

```
// Check if shop session exists
document.cookie.split('; ').find(row => row.startsWith('shop_session='))
```

✗ Failure Scenarios:

- Redirects to error page → Check `SESSION_SECRET` in `.env`
- No cookie set → Check cookie flags (`httpOnly`, `secure`)
- 500 error → Check server logs for JWT encryption errors

Step 2: Sign Up New User

Action:

1. Click “Sign Up” link
2. Navigate to `/signup`
3. Fill form:
 - Email: `newuser+test@jnqlabs.ai`
 - Password: `TestUser123!`
4. Click “Sign Up”

Expected Result:

- ☒ Clerk creates user account
- ☒ Redirects to `/products/qryx/setup`
- ☒ Shop session still valid

Verify in Clerk Dashboard:

1. Go to Clerk Dashboard → Users
2. Find `newuser+test@jnqlabs.ai`
3. User should exist with status “Active”

Database Verification:

```
-- Check user created via webhook
SELECT * FROM users WHERE email = 'newuser+test@jnxlabs.ai';

-- Verify org created
SELECT o.* FROM orgs o
JOIN users u ON u.org_id = o.id
WHERE u.email = 'newuser+test@jnxlabs.ai';
```

✗ Failure Scenarios:

- Signup fails → Check Clerk API keys in .env
- No redirect → Check Clerk redirect URLs configuration
- User not in DB → Check webhook handler logs
- Shop session lost → Check cookie persistence

Step 3: Select Pricing Plan**Action:**

1. On `/products/qryx/setup` page
2. Review all 3 plans:
 - Starter (\$29/month)
 - Professional (\$79/month)
 - Business (\$199/month)
3. Click “Get Started” on **Starter** plan

Expected Result:

- ☒ Redirects to Stripe Checkout
- ☒ Checkout session shows correct plan (\$29/month)
- ☒ Shop domain passed as metadata

Verify Checkout Session:

1. Check URL: `https://checkout.stripe.com/c/pay/...`
2. Plan shows: “Qryx Starter - \$29/month”
3. Total: \$29.00 USD

DevTools Network Tab:

1. Find `POST /api/stripe/checkout` request
2. Check Request Body:

```
json
{
  "planId": "starter",
  "shop": "shopbotv3.myshopify.com"
}
```

3. Check Response:

```
json
{
  "sessionId": "cs_test_...",
  "url": "https://checkout.stripe.com/..."
}
```

Stripe Dashboard Verification:

1. Go to Stripe Dashboard → Payments → Checkout Sessions
2. Find recent session
3. Verify metadata contains `shop: shopbotv3.myshopify.com`

✗ Failure Scenarios:

- 400 Bad Request → Shop session expired (restart from Step 1)
 - 500 Server Error → Check Stripe API keys
 - Wrong plan shown → Check Price IDs in `.env`
 - No shop metadata → Check checkout API implementation
-

Step 4: Complete Stripe Payment**Action:**

1. On Stripe Checkout page
2. Fill payment form:
 - Email: Same as signup (`newuser+test@jnxlabs.ai`)
 - Card: `4242 4242 4242 4242`
 - Expiry: `12/26`
 - CVC: `123`
 - Name: `Test User`
 - ZIP: `12345`
3. Click "Subscribe"

Expected Result:

- ☒ Payment processes successfully
- ☒ Redirects to `/api/qryx/callback?shop=...`
- ☒ Then redirects to Shopify OAuth screen

Stripe Dashboard Verification:

1. Go to **Payments** → Find recent payment
2. Status should be "Succeeded"
3. Go to **Customers** → Find customer
4. Go to **Subscriptions** → Should show active subscription

Webhook Verification:

1. Go to **Developers** → **Webhooks** → **Your Endpoint**
2. Check **Recent deliveries**
3. Should see `checkout.session.completed` event
4. Status: 200 OK (green checkmark)
5. Click event → View response body

Database Verification:

```
-- Check subscription created
SELECT * FROM billing_subscriptions
WHERE clerk_user_id = (SELECT clerk_user_id FROM users WHERE email = 'newuser+test@jnx
labs.ai')
ORDER BY created_at DESC LIMIT 1;

-- Expected fields:
-- plan_id: 'starter'
-- status: 'active'
-- shop_domain: 'shopbotv3.myshopify.com'
-- stripe_subscription_id: 'sub_...'
-- current_period_end: ~30 days from now
```

✗ Failure Scenarios:

- Payment declined → Use test card 4242 4242 4242 4242
- Webhook not fired → Check webhook configuration
- No DB record → Check webhook handler logs
- Wrong shop domain → Check session extraction in callback
- 404 after payment → Check OAuth callback URL

Step 5: Shopify OAuth

Action:

1. After Stripe redirect, arrive at Shopify OAuth screen
2. Screen shows: “Qryx wants to access your store”
3. Review permissions requested
4. Click “Install App”

Expected Result:

- ☒ OAuth completes successfully
- ☒ Redirects to `/app/products/qryx` (Qryx Dashboard)
- ☒ Shop access token stored in database

Database Verification:

```
-- Check Shopify shop record
SELECT * FROM qryx_shops WHERE shop_domain = 'shopbotv3.myshopify.com';

-- Expected fields:
-- shop_domain: 'shopbotv3.myshopify.com'
-- access_token: Encrypted token
-- scope: 'read_products,read_customers,...'
-- status: 'active'
```

✗ Failure Scenarios:

- OAuth fails → Check Shopify API credentials
- Redirect mismatch → Verify redirect URI in Shopify Partner Dashboard
- Access token not saved → Check OAuth callback handler
- 500 error → Check database schema for qryx_shops table

Step 6: Access Qryx Dashboard

Action:

1. Should automatically land on `/app/products/qryx`
2. Dashboard shows:
 - Shop: shopbotv3.myshopify.com
 - Plan: Starter
 - Status: Active
 - Conversations: 0 / 500

Expected Result:

- ☒ Dashboard loads without errors
- ☒ Correct shop information displayed
- ☒ Plan details match subscription
- ☒ Widget configuration available

Verify API Endpoints:

1. Get Configuration:

```
curl -X GET https://www.jnxlabs.ai/api/qryx/config?shop=shopbotv3.myshopify.com
```

Expected Response:

```
{
  "shop": "shopbotv3.myshopify.com",
  "chatbotConfig": {
    "enabled": true,
    "theme": {...},
    "behavior": {...}
  },
  "subscription": {
    "plan": "starter",
    "status": "active",
    "conversationLimit": 500,
    "conversationCount": 0
  }
}
```

2. Test Chat:

```
curl -X POST https://www.jnxlabs.ai/api/qryx/chat \
-H "Content-Type: application/json" \
-d '{
  "shop": "shopbotv3.myshopify.com",
  "message": "Hello, what products do you have?",
  "conversationId": "test-123"
}'
```

Expected Response:

```
{
  "response": "Hi! Welcome to our store...",
  "conversationId": "test-123"
}
```

✗ Failure Scenarios:

- Dashboard shows wrong plan → Check database subscription record
- 404 on API calls → Check shop authentication
- Chat not working → Check Gemini API key
- Usage not tracked → Check conversation counter logic

3. Test Scenario 2: Existing User - Return Flow

Goal: Verify flow for user who already has account but wants to add Qryx to new shop.

Step 1: Start Installation

Action:

1. Open browser in **Incognito Mode**
2. Navigate to: `https://www.jnqlabs.ai/api/qryx/install?shop=newshop.myshopify.com`

Expected Result:

- ✓ Redirects to `/login`
- ✓ Shop session created for `newshop.myshopify.com`

Step 2: Login Existing User

Action:

1. On `/login` page
2. Enter credentials:
 - Email: `newuser+test@jnqlabs.ai`
 - Password: `TestUser123!`
3. Click "Sign In"

Expected Result:

- ✓ Login successful
- ✓ Redirects to `/products/qryx/setup`
- ✓ Shop session preserved (`newshop.myshopify.com`)

✗ Failure Scenarios:

- Wrong shop shown → Check session extraction logic
- Login fails → Check credentials
- No redirect → Check Clerk configuration

Step 3-6: Complete Flow

Repeat Steps 3-6 from Test Scenario 1, verifying:

- User can add subscription for new shop

- Previous shop subscription unaffected
- Database shows multiple subscriptions for same user

Database Verification:

```
-- Should show 2 subscriptions
SELECT shop_domain, plan_id, status FROM billing_subscriptions
WHERE clerk_user_id = (SELECT clerk_user_id FROM users WHERE email = 'newuser+test@jnx
labs.ai')
ORDER BY created_at;

-- Expected result:
-- shopbotv3.myshopify.com | starter | active
-- newshop.myshopify.com   | starter | active
```

4. Test Scenario 3: Shop Session Expiry

Goal: Verify error handling when shop session expires.

Step 1: Initiate Installation

Action:

1. Navigate to install URL: <https://www.jnxxlabs.ai/api/qryx/install?shop=testshop.myshopify.com>
2. Note the time (shop session expires in 30 minutes)

Step 2: Wait or Manually Expire Session

Option A: Wait 30 Minutes

- Let session naturally expire

Option B: Manual Expiry (Developer)

```
// In DevTools Console, delete shop_session cookie
document.cookie = 'shop_session=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
```

Step 3: Attempt to Continue

Action:

1. Login/Signup
2. Try to select plan

Expected Result:

- ☒ Error message: "Shop session expired. Please restart installation."
- ☒ Link to restart: Redirects to Shopify App Listing
- ☒ No Stripe checkout created

✗ Failure Scenarios:

- Checkout succeeds without shop → Critical bug! Shop must be required.

- No error shown → Add better error handling
 - Server error → Add try-catch blocks
-

5. Test Scenario 4: Payment Failure Handling

Goal: Verify system handles failed payments gracefully.

Step 1-3: Complete Up to Stripe Checkout

Follow Test Scenario 1, Steps 1-3.

Step 4: Use Declining Test Card

Action:

1. On Stripe Checkout page
2. Use declining card: 4000 0000 0000 0002
3. Fill other fields normally
4. Click "Subscribe"

Expected Result:

- ☒ Stripe shows error: "Your card was declined"
- ☒ User remains on checkout page
- ☒ Can retry with different card
- ☒ No database record created
- ☒ No OAuth initiated

Verify No Side Effects:

```
-- Should return 0 rows
SELECT * FROM billing_subscriptions
WHERE shop_domain = 'shopbotv3.myshopify.com'
AND created_at > NOW() - INTERVAL '5 minutes';
```

Step 5: Retry with Valid Card

Action:

1. On same checkout page
2. Use valid card: 4242 4242 4242 4242
3. Complete payment

Expected Result:

- ☒ Payment succeeds
 - ☒ Flow continues normally to OAuth
 - ☒ Subscription created in database
-

6. Test Scenario 5: Webhook Failure & Retry

Goal: Verify system handles webhook failures and Stripe's automatic retries.

Simulate Webhook Failure

Method 1: Temporarily Break Database

1. In `/api/stripe/webhook/route.ts`, add temporary error:

```
if (event.type === 'checkout.session.completed') {
  throw new Error('Simulated webhook failure');
}
```

1. Complete a payment
2. Verify webhook fails (Stripe Dashboard shows 500 error)
3. Remove error code
4. In Stripe Dashboard, click "Retry" on failed webhook
5. Verify subscription now created

Method 2: Use Stripe CLI

```
# Send test webhook
stripe trigger checkout.session.completed

# Verify handling
# Check server logs
# Check database for new record
```

7. End-to-End Testing Checklist

Complete Flow Verification

- [] **Step 1:** Install URL creates shop session
- [] **Step 2:** Login/Signup preserves shop session
- [] **Step 3:** Plan selection page loads correctly
- [] **Step 4:** Stripe checkout created with metadata
- [] **Step 5:** Payment processing completes
- [] **Step 6:** Webhook fires and creates DB record
- [] **Step 7:** OAuth redirect includes shop parameter
- [] **Step 8:** OAuth completes and stores token
- [] **Step 9:** Dashboard shows correct data
- [] **Step 10:** Chat API works

Edge Cases

- [] Shop session expiry handled
- [] Payment failure handled
- [] Webhook retry works
- [] Duplicate subscription prevented

- [] Multiple shops per user supported
- [] Invalid shop domain rejected
- [] Missing parameters handled gracefully

Security

- [] Shop session JWT encrypted
- [] Webhook signature verified
- [] API keys not exposed to client
- [] HTTPS enforced on all endpoints
- [] CORS configured correctly
- [] Rate limiting active (if implemented)

Performance

- [] Install to login < 2 seconds
- [] Plan selection page < 1 second
- [] Stripe checkout creation < 3 seconds
- [] Webhook processing < 1 second
- [] OAuth callback < 2 seconds
- [] Dashboard load < 3 seconds

8. Monitoring & Debugging

Server Logs

Development:

```
# Terminal running yarn dev shows all logs
cd /home/ubuntu/jnx-os/nextjs_space
yarn dev

# Watch for:
# ✓ [API] /api/qryx/install
# ✓ [API] /api/stripe/checkout
# ✓ [API] /api/stripe/webhook
# ✓ [API] /api/qryx/callback
```

Production (Vercel):

```
# Install Vercel CLI
npm i -g vercel

# Login
vercel login

# View logs
vercel logs www.jnxlabs.ai --follow
```

Database Queries

Check Recent Subscriptions:

```

SELECT
  bs.shop_domain,
  bs.plan_id,
  bs.status,
  bs.stripe_subscription_id,
  bs.created_at,
  u.email as user_email
FROM billing_subscriptions bs
JOIN users u ON u.clerk_user_id = bs.clerk_user_id
ORDER BY bs.created_at DESC
LIMIT 10;

```

Check Active Subscriptions:

```

SELECT
  plan_id,
  COUNT(*) as count
FROM billing_subscriptions
WHERE status = 'active'
GROUP BY plan_id;

```

Check Failed Webhooks:

```

SELECT * FROM system_events
WHERE event_type = 'webhook_failed'
AND created_at > NOW() - INTERVAL '1 hour'
ORDER BY created_at DESC;

```

Stripe Dashboard

Check Checkout Sessions:

1. **Payments → Checkout Sessions**
2. Filter by “Last 24 hours”
3. Look for:
 - Status: “complete”
 - Metadata: shop field present
 - Customer email matches user

Check Webhooks:

1. **Developers → Webhooks → Your Endpoint**
2. **Recent deliveries** tab
3. Look for:
 - Green checkmarks (200 OK)
 - Response time < 1s
 - No failed deliveries

Check Subscriptions:

1. **Customers → All Customers**
2. Find recent customer
3. Verify:
 - Active subscription exists
 - Plan matches checkout
 - Billing cycle correct

Browser DevTools

Network Tab:

- Monitor all API calls
- Check request/response bodies
- Verify status codes (200, 302)
- Check timing for performance

Console Tab:

- Watch for JavaScript errors
- Check custom log statements
- Verify no CORS errors

Application Tab:

- Check cookies (shop_session)
- Verify Local Storage (if used)
- Check Session Storage

9. Automated Testing (Future)

Playwright E2E Tests (Not Yet Implemented)

Example Test Structure:

```
// tests/e2e/qryx-installation.spec.ts
import { test, expect } from '@playwright/test';

test('complete qryx installation flow', async ({ page }) => {
  // Step 1: Initiate install
  await page.goto('https://www.jnslabs.ai/api/qryx/install?shop=test.myshopify.com');
  await expect(page).toHaveURL(/\/login/);

  // Step 2: Sign up
  await page.fill('[name="email"]', 'test@example.com');
  await page.fill('[name="password"]', 'TestPass123!');
  await page.click('button[type="submit"]');

  // Step 3: Select plan
  await expect(page).toHaveURL(/\/products\/qryx\/setup/);
  await page.click('button:has-text("Get Started"):first');

  // Step 4: Complete Stripe checkout
  // (Would need Stripe test fixtures)

  // Step 5: Verify dashboard
  await expect(page).toHaveURL(/\/app\/products\/qryx/);
  await expect(page.locator('text=test.myshopify.com')).toBeVisible();
});
```

Run Tests:

```
# Install Playwright
yarn add -D @playwright/test

# Run tests
yarn playwright test
```

10. Success Criteria

All Tests Must Pass

- ✓ **Test Scenario 1** (New User): Complete flow works end-to-end
- ✓ **Test Scenario 2** (Existing User): Return flow works correctly
- ✓ **Test Scenario 3** (Session Expiry): Error handling works
- ✓ **Test Scenario 4** (Payment Failure): Graceful failure & retry
- ✓ **Test Scenario 5** (Webhook Retry): Automatic retry succeeds

Zero Critical Bugs

- No data loss scenarios
- No security vulnerabilities
- No broken user flows
- No unhandled errors

Performance Targets Met

- All page loads < 3 seconds
- API responses < 1 second
- Webhook processing < 1 second

Monitoring Operational

- Stripe webhook delivery 100%
 - Database records consistent
 - Error logs clean
 - No failed payments without user action
-

11. Reporting Issues

Issue Template

```

**Test Scenario:** [e.g., Test Scenario 1, Step 4]
**Expected Result:** [What should happen]
**Actual Result:** [What actually happened]
**Steps to Reproduce:**
1. ...
2. ...
3. ...

**Environment:**
- URL: [Dev/Production]
- Browser: [Chrome 120, Firefox 121, etc.]
- User: [test@example.com]
- Shop: [test.myshopify.com]

**Logs:**
[Paste relevant server logs, browser console errors]

**Screenshots:**
[Attach screenshots if visual issue]

**Database State:**
[Relevant SQL query results]

**Priority:** [Critical/High/Medium/Low]

```

Where to Report

- **GitHub Issues:** <https://github.com/JNXLabs/jnx-os/issues>
- **Internal Slack:** #qryx-bugs channel
- **Email:** engineering@jnxlabs.ai

Summary

This testing guide covers:

- ✓ **5 Test Scenarios:** From happy path to edge cases
- ✓ **10-Step Verification:** Complete end-to-end flow
- ✓ **Edge Case Coverage:** Session expiry, payment failures, webhook retries
- ✓ **Monitoring Tools:** Logs, database queries, Stripe dashboard
- ✓ **Success Criteria:** Clear definition of “done”

Next Steps After Testing:

1. ✓ Fix any bugs found
2. ✓ Re-test failed scenarios
3. ✓ Document any new edge cases
4. ✓ Update troubleshooting guide
5. ✓ Prepare for Production deployment
6. ✓ Schedule Phase 5B implementation

Document Version: 1.0

Last Updated: December 29, 2025

Maintained By: JNXLabs QA Team